Sistema Especialista de Recomendação de Dieta

Introdução à Inteligência Artificial

Universidade Estadual de Maringá — Departamento de Informática

- **Professor:** Wagner Igarashi
- **Alunos:** Caetano Vendrame Mantovani, Lorenzo Henrique Zanetti, Vitor da Rocha Machado
- **Linguagem:** Prolog (SWI-Prolog) + Python

Introdução

> **Como recomendar uma dieta alimentar adequada a um usuário**, considerando seus objetivos, condições de saúde, restrições alimentares, tempo de preparo e orçamento, de forma explicável?

Por que um Sistema Especialista?

- Várias dietas podem ser plausíveis para o mesmo perfil.
- Algumas condições clínicas apresentam **contraindicações** estritas.
- Outras condições fornecem apenas **peso adicional** à recomendação.
- A entrevista pode ser conduzida com perguntas condicionais e eficientes.
- O sistema orienta o usuário, lembrando-o de consultar um especialista.

Modelagem do Problema

Entidades principais

- **Dieta**: hipótese a ser confirmada ou descartada — 11 dietas modeladas.
- **Paciente**: indivíduo com perfil dinâmico de atributos.
- **Evidência**: condição que favorece uma dieta (peso heurístico).
- **Contraindicação**: condição que exclui uma dieta por razão clínica estrita.

Atributos do perfil

- objetivo
- nível de atividade
- faixa etária
- IMC
- doenças
- restrições alimentares
- tempo de preparo
- orçamento

Modelagem do Problema: fórmula de score

O score de cada dieta é calculado como:

$$\text{Score}(d, x) = \min\left(0.99,\ P_{\text{base}}(d) + \sum_{e \in E(d,x)} \text{peso}(e)\right)$$

Onde:

- $P_{\text{base}}(d)$ é a probabilidade a priori da dieta d
- $E(d, x)$ é o conjunto de evidências satisfeitas pelo perfil x
- O limite de 0.99 evita certeza absoluta

Tabela de Dietas

ID	Nome	Prob. Base	Descrição resumida
low_carb	Low Carb	0.50	Redução de carboidratos; eficaz para perda de peso e controle glicêmico.
vegetariana	Vegetariana	0.50	Exclui carnes, permite laticínios e ovos. Rica em fibras.
vegana	Vegana	0.45	Exclusivamente vegetal; exige suplementação de B12.
mediterranea	Mediterrânea	0.55	Alimentos frescos, azeite, peixes. Forte evidência cardiovascular e metabólica.
hiperproteica	Hiperproteica	0.50	Alta proteína (1.6–2.2 g/kg); indicada para hipertrofia e prevenção de sarcopenia.

Tabela de Dietas

ID	Nome	Prob. Base	Descrição resumida
low_fat	Low Fat	0.45	Redução de gorduras; indicada para colesterol e perfis sedentários.
sem_gluten	Sem Glúten	0.40	Exclusão de trigo, centeio e cevada. Obrigatória para celíacos.
dash	DASH	0.45	Desenvolvida para hipertensão e síndrome metabólica; restrição de sódio.
cetogenica	Cetogênica	0.40	Induz cetose; potente para perda de peso e controle glicêmico tipo 2.
paleolitica	Paleolítica	0.45	Carnes, sementes e raízes; exclui grãos e processados.
flexitariana	Flexitariana	0.55	Majoritariamente vegetal com consumo esporádico de carnes.

Fundamentação Teórica

Sistemas Especialistas

- Aplicam conhecimento explícito de um domínio em forma de **fatos e regras**.
- Separam a **base de conhecimento** do **motor de inferência** (Russell & Norvig, 2022).
- Oferecem **explicabilidade**: o sistema pode justificar cada decisão tomada.

Por que Prolog?

- A resolução SLD do Prolog realiza **encadeamento para trás nativamente**.
- Fatos e regras são representados uniformemente como cláusulas de Horn.
- `findall/3`, `assert/retract` e `msort/2` tornam o motor conciso e expressivo.

Fundamentação Teórica: Encadeamento

← Encadeamento para trás

(Backward Chaining)

Parte de uma **hipótese** e busca fatos que a provem.

> "Esta dieta tem score alto?" → verifica cada regra de evidência → consulta o perfil do usuário.

Usado em: `calcular_score`, `contraindicada`, `recomendar`.

Encadeamento para frente →

(Forward Chaining)

Parte de **fatos conhecidos** e deriva novas conclusões.

> Usuário responde `restricao_carne = sim` → sistema avança e ativa `restricao_laticinios`.

Usado em: `listar_perguntas_condicionais_ativas`.

Fundamentação Teórica: como os dois coexistem

O sistema opera em **duas fases complementares**:

1. **Coleta de fatos** — conduzida por **encadeamento para frente**: respostas do usuário disparam novas perguntas condicionais até o perfil estar completo.
2. **Inferência de recomendações** — conduzida por **encadeamento para trás**: o motor parte das hipóteses (dietas) e consulta o perfil para calcular scores e contraindicações.

Essa separação é um padrão clássico de sistemas especialistas (Russell & Norvig, cap. 9) e permite que a entrevista seja eficiente sem comprometer a profundidade do raciocínio.

Fundamentação Teórica: probabilidade heurística

- O modelo de probabilidade é **heurístico**, não bayesiano estrito.
- A soma de evidências simula um acúmulo de suporte para cada hipótese.
- Contraindicações são tratadas como **regras booleanas duras** — se ativadas, excluem a dieta independente do score.
- Essa abordagem é análoga ao modelo de **certeza acumulativa** dos sistemas especialistas clássicos (Shortliffe, MYCIN, 1976).

Materiais e Métodos

Ambiente e ferramentas

- **Sistema operacional:** Windows 10/11
- **Linguagens:** SWI-Prolog 9.x e Python 3.11+
- **Biblioteca de integração:** `pyswip` (ponte Python ↔ SWI-Prolog via `subprocess`)
- **Framework de testes:** `plunit` (nativo do SWI-Prolog)

Execução

```
# Sistema principal
python interface.py

# Testes unitários
swipl -q -g "executar_testes_sistema" -t halt testes_unitarios.pl
```

Materiais e Métodos: arquitetura de arquivos

```
sistema-especialista-dieta/
├── base_conhecimento.pl   ← dietas, perguntas, evidências, contraindicações
├── motor_inferencia.pl    ← contexto do indivíduo, inferência, explicabilidade
├── interface.py           ← menu interativo, integração Python/Prolog, CRUD
├── testes_unitarios.pl    ← 6 suítes de testes com plunit
└── slides.md              ← esta apresentação
```

Fluxo de dados

`interface.py` carrega `motor_inferencia.pl`, que consulta `base_conhecimento.pl`.

Respostas do usuário são `assert`adas dinamicamente no Prolog em tempo de execução.

Desenvolvimento: base de conhecimento

- **11 dietas** declaradas com `dieta(Id, Nome, ProbBase)` e descrições textuais.
- **15 perguntas** com `pergunta/5`: atributo, tipo, texto, opções e justificativa.
- **Perguntas base**: sempre apresentadas.
- **Perguntas condicionais**: ativadas dinamicamente por respostas anteriores.

```
dieta(dash, 'DASH', 0.45).

pergunta(tipo_diabetes, depende(tem_diabetes, sim),
    'Qual e o tipo do seu diabetes?',
    [tipo_1, tipo_2, pre_diabetes],
    'Tipo clinico altera seguranca e conduta nutricional.').
```

Desenvolvimento: perguntas condicionais

Atributo	Depende de	Texto
frequencia_carne	restricao_carne = nao	Você consome carne com qual frequência semanal?
restricao_laticinios	restricao_carne = sim	Também restringe laticínios e ovos?
disposicao_restricao	objetivo = perda_peso	Aceita cortar grupos alimentares de forma intensa?
tipo_diabetes	tem_diabetes = sim	Qual é o tipo do seu diabetes?

A ativação condicional é **encadeamento para frente** em ação: o fato `restricao_carne = sim` propaga para frente e faz surgir `restricao_laticinios`.

Desenvolvimento: modelagem do perfil

- Respostas armazenadas como predicados dinâmicos:
  - `tem_objetivo/2`, `tem_nivel_atividade/2`, `tem_faixa_etaria/2`, etc.
  - `tem_doenca/2` / `nao_tem_doenca/2`
  - `tem_restricao/2` / `nao_tem_restricao/2`
- O indivíduo ativo é controlado por `paciente_atual/1`.
- `limpar_respostas/1` reseta o perfil entre sessões.
- `registrar_resposta/3` mantém consistência: ao responder `tem_diabetes = nao`, o predicado `tem_tipo_diabetes` é removido automaticamente.

Desenvolvimento: regras de evidência

```
evidencia(low_carb,     X, 0.25) :- tem_objetivo(X, perda_peso).
evidencia(dash,         X, 0.30) :- tem_doenca(X, hipertensao).
evidencia(dash,         X, 0.12) :- tem_doenca(X, diabetes).
evidencia(mediterranea, X, 0.10) :- tem_doenca(X, diabetes).
evidencia(hiperproteica,X, 0.08) :- tem_faixa_etaria(X, idoso).
evidencia(vegana,       X, 0.10) :- tem_imc_faixa(X, obesidade).
```

Cada chamada é resolvida por **encadeamento para trás**: o motor parte da hipótese (a dieta) e verifica se o perfil satisfaz a condição declarada.

Desenvolvimento: regras de contraindicação

- `contraindicada(Dieta, X)` é uma **regra booleana estrita**.
- Dietas contraindicadas são completamente excluídas do ranking.

```
contraindicada(cetogenica, X) :- tem_doenca(X, colesterol_alto).
contraindicada(cetogenica, X) :- tem_doenca(X, doenca_cardiaca).
contraindicada(cetogenica, X) :- tem_tipo_diabetes(X, tipo_1).
contraindicada(cetogenica, X) :- tem_imc_faixa(X, abaixo_peso).
contraindicada(hiperproteica,X):- tem_doenca(X, problemas_renais).
contraindicada(vegetariana,  X):- tem_restricao(X, laticinios).
```

Tabela de Contraindicações

Dieta	Condição	Justificativa
cetogenica	disposicao_restricao = nao	Muito restritiva; exige alta adesão.
cetogenica	colesterol_alto	Incompatível com altíssima ingestão de gordura.
cetogenica	doenca_cardiaca	Risco cardiovascular elevado.
cetogenica	tipo_diabetes = tipo_1	Requer controle glicêmico estável.
cetogenica	imc_faixa = abaixo_peso	Risco de déficit calórico e perda muscular.
hiperproteica	restricao_carne	Baseia-se fundamentalmente em carnes.
hiperproteica	problemas_renais	Sobrecarrega a função renal.
vegetariana	restricao_laticinios	Depende de laticínios; inviável com essa restrição.
vegana	tempo_preparo = baixa	Planejamento de proteínas vegetais é essencial.
mediterranea	orcamento = baixo	Ingredientes frescos têm custo elevado.
paleolitica	problemas_renais	Alta proteína sobrecarrega os rins.
low_fat	objetivo = ganho_massa	Impede síntese hormonal necessária ao ganho muscular.

Desenvolvimento: motor de inferência

```
calcular_score(Dieta, X, Score) :-
    dieta(Dieta, _, ProbBase),
    findall(Peso, evidencia(Dieta, X, Peso), Pesos),
    sum_list(Pesos, Soma),
    Score is min(0.99, ProbBase + Soma).

recomendar(X, Lista) :-
    findall(Score-Dieta-Nome, (
        dieta(Dieta, Nome, _),
        nao_contraindicada(Dieta, X),
        calcular_score(Dieta, X, Score)
    ), Unsorted),
    msort(Unsorted, Sorted),
    reverse(Sorted, Lista).
```

Desenvolvimento: explicabilidade

O sistema responde a três perguntas:

- **"Por que esta dieta foi recomendada?"**
  → `fatos_confirmados/3` lista os pesos que contribuíram ao score.
- **"Por que esta dieta foi descartada?"**
  → `razoes_exclusao/3` lista as contraindicações ativadas.
- **"Por que foi perguntado sobre X?"**
  → `detalhes_pergunta/4` retorna a justificativa declarada em `pergunta/5`.

```
fatos_confirmados(Dieta, X, Pesos) :-
    findall(Peso, evidencia(Dieta, X, Peso), Pesos).
```

Desenvolvimento: interface e CRUD

- `interface.py` usa `pyswip` para invocar SWI-Prolog embutido no Python.
- O questionário aplica **forward chaining**: após cada resposta, checa quais condicionais foram ativadas.
- O CRUD parseia `base_conhecimento.pl` com expressões regulares e permite **C**riar, **R**eadsitar, **A**lterar e **E**xcluir dietas, evidências e contraindicações em tempo de execução.
- Após qualquer modificação, o Prolog é recarregado automaticamente.

Desenvolvimento: testes unitários

- `testes_unitarios.pl` organiza **6 suítes** com `plunit`, cobrindo todos os componentes.

Suíte	O que valida
`calculo_score_lpo`	score base, crescimento por evidência, limite 0.99
`contraindicacoes_lpo`	cada regra de exclusão estrita
`recomendacoes_lpo`	ordenação, ausência de dietas bloqueadas, consistência
`perguntas_condicionais_lpo`	ativação e não-repetição de condicionais
`modelo_relacional_lpo`	consistência do armazenamento dinâmico
`explicabilidade_lpo`	`fatos_confirmados`, `razoes_exclusao`, `detalhes_pergunta`

Desenvolvimento: fluxo completo

```
Usuário inicia sessão
        │
        ▼
  Perguntas base ──────────────────────────────────────────────────┐
        │                                                          │
        │   resposta nova    ← FORWARD CHAINING →                  │
        ▼                                                          │
  Ativa condicionais? ─── sim ──► faz pergunta adicional ──────────┘
        │ não
        ▼
  Perfil completo → recomendar(X, Lista)   ← BACKWARD CHAINING
        │
        ▼
  Lista ordenada por score decrescente
        │
        ▼
  Usuário consulta explicações (por que? / por que não?)
```

Conclusões

- O sistema combina **encadeamento para frente** na condução da entrevista e **encadeamento para trás** na inferência — arquitetura híbrida clássica de sistemas especialistas.
- As evidências refletem recomendações clínicas atuais: diabetes para DASH e Mediterrânea, prevenção de sarcopenia para hiperproteica em idosos, risco metabólico para cetogênica em abaixo do peso.
- A **explicabilidade** permite que o usuário entenda o porquê de cada recomendação e exclusão.
- O CRUD em tempo de execução torna a base extensível sem recompilação.
- Os 6 conjuntos de testes unitários garantem cobertura de todos os componentes críticos.

⚠️ **Este protótipo é apenas informativo. Consulte um nutricionista ou médico para recomendação correta e precisa.**

Referências Bibliográficas

- RUSSELL, S.; NORVIG, P. *Inteligência Artificial: Uma Abordagem Moderna*. 4. ed. Pearson, 2022.
- BRATKO, I. *Prolog Programming for Artificial Intelligence*. 4. ed. Pearson, 2011.
- CLOCKSIN, W. F.; MELLISH, C. S. *Programming in Prolog*. 5. ed. Springer, 2003.
- SWI-PROLOG. *SWI-Prolog Reference Manual*. Disponível em: https://www.swi-prolog.org/pldoc/. Acesso em: abr. 2026.
- SHORTLIFFE, E. H. *Computer-Based Medical Consultations: MYCIN*. Elsevier, 1976.
- ORNISH, D. et al. *Intensive Lifestyle Changes for Reversal of Coronary Heart Disease*. JAMA, 1998.
- SACKS, F. M. et al. *Effects on Blood Pressure of Reduced Dietary Sodium and the DASH Diet*. NEJM, 2001.